NATIONAL ECONOMICS UNIVERSITY

School of Advanced Education Program

-----*-----**

MIDTERM 1 REPORT

Exercise 3: Electronics Repair Management

MySQL database electronics_mini

Group 4

Full name	Student ID
Đỗ Thu Uyên	11230785
Nguyễn Vũ Ngọc Châu	11230738
Trịnh Gia Khánh	11230753

Course: IST2610 - Database Management in Business

Lecturer: Dr. Tran Duc Minh

Hanoi, November 2026

Table of Contents

1. Introduction **3**
- 1.1. Context 3
- 1.2. Objectives 3
- 1.3. Deliverables and environment 3
- 1.4. Work allocation 4

2. Database Design **4**
- 2.1. Entity-relationship diagram 4
- 2.2. Table specifications 5
- 2.3. Cross-table business rules 7
- 2.4. Requirement compliance 7
- 2.5. Constraint violation tests 8

3. Sample data **9**
- 3.1. Record counts 9
- 3.2. Scenario matrix 10
- 3.3. Additional edge cases 10
- 3.4. Validation results 11
- 3.5. Data modification test 12

4. Queries **13**
- 4.1. Query 1: Completed orders in January 2025 13
- 4.2. Query 2: Orders per specialty in Q1 2025 14
- 4.3. Query 3: Customers with an order lacking a report 15
- 4.4. Query 4: Average repair cost per specialty 16
- 4.5. Query 5: Top 3 repairers in 2025 17
- 4.6. Query 6: Total paid per customer in 2025 18
- 4.7. Query 7: Customers with no payment record 20
- 4.8. Query 8: Customers by total confirmed cost 20
- 4.9. Query 9: Cancellation rate per repairer in 2025 22
- 4.10. Query 10: Customers with more than 3 Completed orders in 2025 23

5. Views **24**
- 5.1. v_detail 24
- 5.2. v_group_summary 25
- 5.3. v_debt 27
- 5.4. The double-counting pitfall 28

6. Conclusion **29**

1. Introduction

1.1. Context

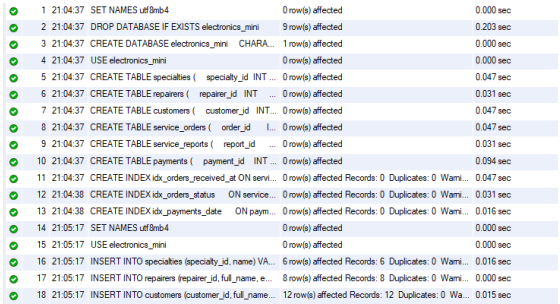
The case study concerns an electronics repair shop that services computers, mobile phones and other electronic devices. The shop requires a system for managing its technical staff, service orders, repair outcomes and customer payments. Each service order records a single device (identified by `device_name` and `serial_number`) and is assigned to a single repairer, who is responsible for the entire order. The management of spare parts and the reassignment of staff during a repair fall outside the scope of this exercise.

1.2. Objectives

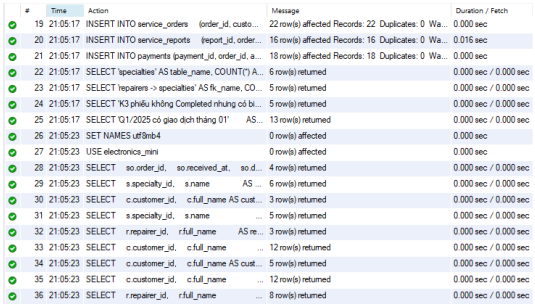
- To design a relational database for repair management and implement it using Data Definition Language (DDL) statements.
- To populate the database with sample data using Data Manipulation Language (DML) statements and to validate the data through integrity-check queries.
- To apply `JOIN`, `GROUP BY` and `HAVING` clauses in 10 analytical queries.
- To construct views that present order details, specialty-level statistics and outstanding customer debt.

1.3. Deliverables and environment

The submission comprises four SQL scripts, which must be executed on a newly created database in the following order: `schema.sql`, `seed.sql`, `queries.sql` and `views.sql`. All results reported in this document were obtained on MySQL 8.4.0 under the default `sql_mode`, which includes `STRICT_TRANS_TABLES`. MySQL 8.0.16 or later is required, since earlier versions parse `CHECK` constraints without enforcing them. All personal data in the sample dataset are fictitious.



#	Time	Action	Message	Duration / Fetch
1	21:04:37	SET NAMES utf8mb4	0 row(s) affected	0.000 sec
2	21:04:37	DROP DATABASE IF EXISTS electronics_mini	9 row(s) affected	0.203 sec
3	21:04:37	CREATE DATABASE electronics_mini CHARA...	1 row(s) affected	0.000 sec
4	21:04:37	USE electronics_mini	0 row(s) affected	0.000 sec
5	21:04:37	CREATE TABLE specialties (specialty_id INT ...	0 row(s) affected	0.047 sec
6	21:04:37	CREATE TABLE repairers (repairer_id INT ...	0 row(s) affected	0.031 sec
7	21:04:37	CREATE TABLE customers (customer_id INT...	0 row(s) affected	0.047 sec
8	21:04:37	CREATE TABLE service_orders (order_id I...	0 row(s) affected	0.047 sec
9	21:04:37	CREATE TABLE service_reports (report_id ...	0 row(s) affected	0.031 sec
10	21:04:37	CREATE TABLE payments (payment_id INT ...	0 row(s) affected	0.094 sec
11	21:04:37	CREATE INDEX idx_orders_received_at ON servi...	0 row(s) affected Records: 0 Duplicates: 0 Warni...	0.047 sec
12	21:04:38	CREATE INDEX idx_orders_status ON service...	0 row(s) affected Records: 0 Duplicates: 0 Warni...	0.031 sec
13	21:04:38	CREATE INDEX idx_payments_date ON paym...	0 row(s) affected Records: 0 Duplicates: 0 Warni...	0.016 sec
14	21:05:17	SET NAMES utf8mb4	0 row(s) affected	0.000 sec
15	21:05:17	USE electronics_mini	0 row(s) affected	0.000 sec
16	21:05:17	INSERT INTO specialties (specialty_id, name) VA...	6 row(s) affected Records: 6 Duplicates: 0 Warni...	0.016 sec
17	21:05:17	INSERT INTO repairers (repairer_id, full_name, e...	8 row(s) affected Records: 8 Duplicates: 0 Warni...	0.000 sec
18	21:05:17	INSERT INTO customers (customer_id, full_name...	12 row(s) affected Records: 12 Duplicates: 0 Wa...	0.015 sec



#	Time	Action	Message	Duration / Fetch
19	21:05:17	INSERT INTO service_orders (order_id, custo...	22 row(s) affected Records: 22 Duplicates: 0 Wa...	0.000 sec
20	21:05:17	INSERT INTO service_reports (report_id, order...	16 row(s) affected Records: 16 Duplicates: 0 Wa...	0.016 sec
21	21:05:17	INSERT INTO payments (payment_id, order_id, a...	18 row(s) affected Records: 18 Duplicates: 0 Wa...	0.000 sec
22	21:05:17	SELECT 'specialties' AS table_name, COUNT(*) A...	6 row(s) returned	0.000 sec / 0.000 sec
23	21:05:17	SELECT 'repairers -> specialties' AS fk_name, CO...	5 row(s) returned	0.000 sec / 0.000 sec
24	21:05:17	SELECT 'K3 phiếu không Completed nhưng có bi...	5 row(s) returned	0.000 sec / 0.000 sec
25	21:05:17	SELECT 'Q1/2025 có giao dịch tháng 01' AS...	13 row(s) returned	0.000 sec / 0.000 sec
26	21:05:23	SET NAMES utf8mb4	0 row(s) affected	0.000 sec
27	21:05:23	USE electronics_mini	0 row(s) affected	0.000 sec
28	21:05:23	SELECT so.order_id, so.received_at, so.d...	4 row(s) returned	0.000 sec / 0.000 sec
29	21:05:23	SELECT s.specialty_id, s.name AS ...	6 row(s) returned	0.000 sec / 0.000 sec
30	21:05:23	SELECT c.customer_id, c.full_name AS cust...	3 row(s) returned	0.000 sec / 0.000 sec
31	21:05:23	SELECT s.specialty_id, s.name ...	5 row(s) returned	0.000 sec / 0.000 sec
32	21:05:23	SELECT r.repairer_id, r.full_name AS re...	3 row(s) returned	0.000 sec / 0.000 sec
33	21:05:23	SELECT c.customer_id, c.full_name ...	12 row(s) returned	0.000 sec / 0.000 sec
34	21:05:23	SELECT c.customer_id, c.full_name AS cust...	5 row(s) returned	0.000 sec / 0.000 sec
35	21:05:23	SELECT c.customer_id, c.full_name ...	12 row(s) returned	0.000 sec / 0.000 sec
36	21:05:23	SELECT r.repairer_id, r.full_name ...	8 row(s) returned	0.000 sec / 0.000 sec

✓	37	21:05:23	SELECT c.customer_id, c.full_name A...	1 row(s) returned	0.000 sec / 0.000 sec
✓	38	21:05:41	SET NAMES utf8mb4	0 row(s) affected	0.000 sec
✓	39	21:05:41	USE electronics_mini	0 row(s) affected	0.000 sec
✓	40	21:05:41	CREATE OR REPLACE VIEW v_detail AS SELE...	0 row(s) affected	0.015 sec
✓	41	21:05:41	CREATE OR REPLACE VIEW v_group_summary...	0 row(s) affected	0.000 sec
✓	42	21:05:41	CREATE OR REPLACE VIEW v_debt AS SELEC...	0 row(s) affected	0.016 sec
✓	43	21:05:41	SELECT * FROM v_detail ORDER BY order_...	22 row(s) returned	0.000 sec / 0.000 sec
✓	44	21:05:41	SELECT * FROM v_group_summary ORDER BY ...	6 row(s) returned	0.000 sec / 0.000 sec
✓	45	21:05:41	SELECT * FROM v_debt ORDER BY custo...	12 row(s) returned	0.000 sec / 0.000 sec

1.4. Work allocation

Table 1. Work allocation

Member	Part	Report sections	Queries
Uyên	A - Schema	Section 2 and Sections 4.1-4.3	1, 2, 3
Châu	B - Seed data	Section 3 and Sections 4.4-4.7	4, 5, 6, 7
Khánh	D - Views	Section 5, Sections 4.8-4.10 and report assembly	8, 9, 10

2. Database Design

2.1. Entity-relationship diagram

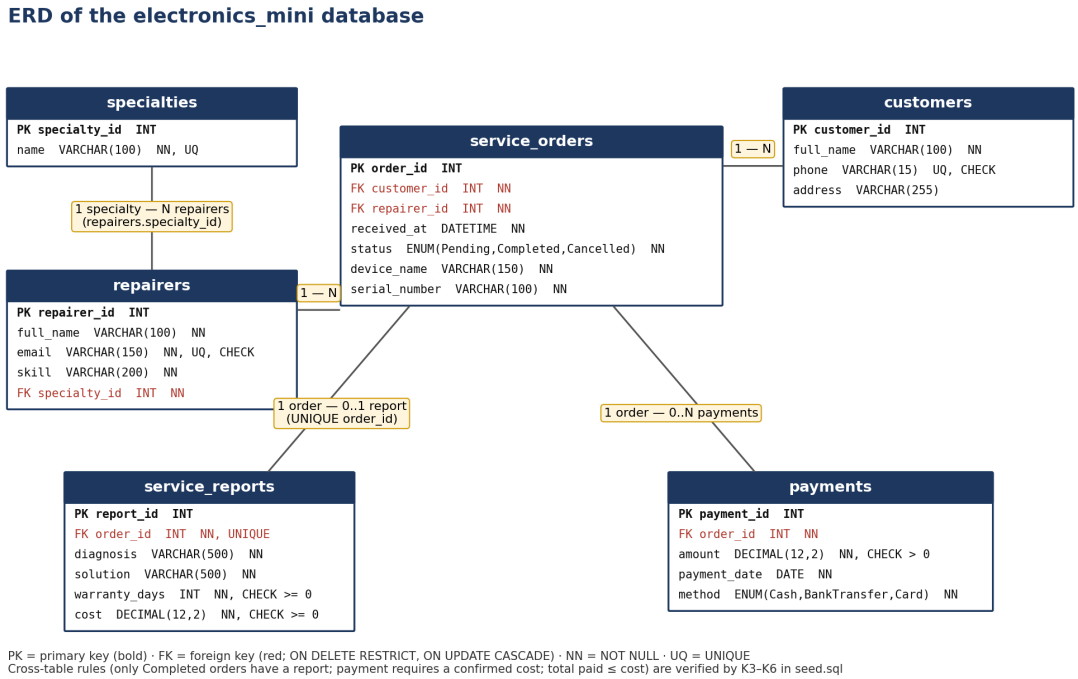


Figure 1. ERD of the `electronics_mini` database

Table 2. Relationships and their implementation

Parent	Child	Cardinality	Implementation
specialties	repairers	1 - N	`specialty_id` NOT NULL + FK: each repairer belongs to exactly one specialty
customers	service_orders	1 - N	`customer_id` NOT NULL + FK: each order belongs to exactly one customer
repairers	service_orders	1 - N	`repairer_id` NOT NULL + FK: each order is assigned to exactly one repairer
service_orders	service_reports	1 - 0..1	`order_id` NOT NULL, UNIQUE + FK: each order has at most one handover report
service_orders	payments	1 - 0..N	`order_id` NOT NULL + FK, not unique: an order may be settled in several instalments

2.2. Table specifications

Table 3. Columns and data types

Table	Column	Type	Constraints
specialties	specialty_id	INT	PK
	name	VARCHAR(100)	NOT NULL, UNIQUE
repairers	repairer_id	INT	PK
	full_name	VARCHAR(100)	NOT NULL
	email	VARCHAR(150)	NOT NULL, UNIQUE, CHECK (format)
	skill	VARCHAR(200)	NOT NULL
	specialty_id	INT	NOT NULL, FK
customers	customer_id	INT	PK

	full_name	VARCHAR(100)	NOT NULL
	phone	VARCHAR(15)	NULL allowed, UNIQUE, CHECK (10 digits starting with 0)
	address	VARCHAR(255)	NULL allowed
service_orders	order_id	INT	PK
	customer_id	INT	NOT NULL, FK
	repairer_id	INT	NOT NULL, FK
	received_at	DATETIME	NOT NULL
	status	ENUM	NOT NULL, DEFAULT Pending; Pending, Completed, Cancelled
	device_name	VARCHAR(150)	NOT NULL
	serial_number	VARCHAR(100)	NOT NULL
service_reports	report_id	INT	PK
	order_id	INT	NOT NULL, UNIQUE, FK
	diagnosis	VARCHAR(500)	NOT NULL
	solution	VARCHAR(500)	NOT NULL
	warranty_days	INT	NOT NULL, DEFAULT 0, CHECK (≥ 0)
	cost	DECIMAL(12,2)	NOT NULL, CHECK (≥ 0)
payments	payment_id	INT	PK
	order_id	INT	NOT NULL, FK
	amount	DECIMAL(12,2)	NOT NULL, CHECK (> 0)
	payment_date	DATE	NOT NULL
	method	ENUM	NOT NULL; Cash, BankTransfer, Card

The principal design decisions are summarised below.

- **DECIMAL for monetary values.** `DECIMAL(12,2)` stores values exactly, whereas floating-point types may introduce rounding errors when amounts are aggregated.

- **DATETIME for `received_at`.** Because orders are received at a specific point in time, all temporal filters employ half-open intervals such as `>= '2025-01-01' AND < '2025-04-01'`. This ensures that records timestamped in the final seconds of a period are not excluded.
- **ON DELETE RESTRICT on all foreign keys.** Payments and handover reports constitute financial records and must therefore be preserved if deletion of the associated order is attempted. `ON UPDATE CASCADE` maintains referential integrity should a key value be modified.
- **Optional customer contact details.** Since the assignment does not specify telephone number or address as mandatory attributes, both columns accept NULL values; customer 10 illustrates this case.
- **Additional constraints.** The UNIQUE and format-validation CHECK constraints on email and telephone number exceed the minimum requirements; they were included because the general rules permit such constraints where appropriate.

2.3. Cross-table business rules

The general rules prohibit `CHECK` constraints that reference other tables and allow such rules to be verified instead through queries in `seed.sql`. As triggers and stored procedures are not required by the assignment, none were implemented.

Table 4. Business rules verified by query

Check	Rule
K3	Only orders with status Completed may have a handover report
K4	Each Completed order has exactly one handover report
K5	Payment may be collected only after the repair cost has been confirmed
K6	The total amount collected for an order does not exceed its cost (payments are first aggregated per order)
K7	Supplementary rule: the payment date is not earlier than the date on which the order was received

2.4. Requirement compliance

Table 5. Part A and general rules compared with `schema.sql`

Requirement	Implementation	Met

Create database `electronics_mini`	`CREATE DATABASE`, utf8mb4	Yes
Six tables containing all required columns	Table and column names match the assignment specification exactly	Yes
The primary key is the leading identifier column	Applied to all six tables	Yes
Reference identifiers are enforced by foreign keys	Five foreign key constraints	Yes
Names, business dates and costs are mandatory	NOT NULL	Yes
Cost is non-negative	`chk_reports_cost`	Yes
`warranty_days` ≥ 0	`chk_reports_warranty`	Yes
`amount` > 0 and valid methods	`chk_payments_amount`, ENUM	Yes
Valid order statuses	ENUM	Yes
UNIQUE `order_id` in `service_reports`	`uq_reports_order`	Yes
DECIMAL for money, DATE or DATETIME for time	`DECIMAL(12,2)`, `DATETIME`, `DATE`	Yes
No CHECK constraint references another table	Cross-table rules are verified by K3-K7	Yes

2.5. Constraint violation tests

Statements V1-V12 are included in `seed.sql` as comments so that the script executes without interruption. For testing purposes, they were uncommented and executed within a `START TRANSACTION ... ROLLBACK` block. Following the rollback, the `payments` table still contained 18 rows, confirming that no test data persisted.

Table 6. Violation test results

Test	Violation	Constraint	Actual result
V1	Order referencing non-existent customer 999	FK	ERROR 1452
V2	Status `'Done'`	ENUM	ERROR 1265
V3	`received_at` is NULL	NOT NULL	ERROR 1048
V4	Second report for order 1	UNIQUE	ERROR 1062
V5	Cost of −100,000	CHECK	ERROR 3819
V6	`warranty_days` of −5	CHECK	ERROR 3819

V7	Amount of 0	CHECK	ERROR 3819
V8	Method `'Momo'`	ENUM	ERROR 1265
V9	Duplicate repairer email	UNIQUE	ERROR 1062
V10	Deletion of customer 1, who has existing orders	FK RESTRICT	ERROR 1451
V11	Payment for Pending order 10	Cross-table	Accepted; K5 = 1
V12	Additional payment of 50,000 on order 1	Cross-table	Accepted; K6 = 1

Violations V1-V10 are rejected by the schema itself. V11 and V12, by contrast, illustrate an inherent limitation of declarative constraints: MySQL accepts both insertions because the corresponding rules depend on data from multiple tables, and the violations are detected only by the check queries. In V12, for instance, order 1 carries a cost of 1,850,000, whereas the amount collected rises to 1,900,000.

#	Time	Action	Message	Duration / Fetch
129	21:57:08	USE electronics_mini	0 row(s) affected	0.016 sec
130	21:57:08	START TRANSACTION	0 row(s) affected	0.000 sec
131	21:57:08	INSERT INTO service_orders VALUES (99, 999, 1, '2025-05-01 09:00:00', 'Pending', 'Laptop test', 'TEST...	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (`electronics_mini`.`servi...	0.000 sec
132	21:57:08	INSERT INTO service_orders VALUES (99, 1, 1, '2025-05-01 09:00:00', 'Done', 'Laptop test', 'TEST-02')	Error Code: 1265. Data truncated for column 'status' at row 1	0.000 sec
133	21:57:08	INSERT INTO service_orders VALUES (99, 1, 1, NULL, 'Pending', 'Laptop test', 'TEST-03')	Error Code: 1048. Column 'received_at' cannot be null	0.000 sec
134	21:57:08	INSERT INTO service_reports VALUES (99, 1, 'Lỗi khác', 'Sửa lại', 30, 500000.00)	Error Code: 1062. Duplicate entry '1' for key 'service_reports.uq_reports_order'	0.000 sec
135	21:57:08	INSERT INTO service_reports VALUES (99, 10, 'Test', 'Test', 30, -100000.00)	Error Code: 3819. Check constraint 'chk_reports_cost' is violated.	0.000 sec
136	21:57:08	INSERT INTO service_reports VALUES (99, 10, 'Test', 'Test', -5, 100000.00)	Error Code: 3819. Check constraint 'chk_reports_warranty' is violated.	0.000 sec
137	21:57:08	INSERT INTO payments VALUES (99, 1, 0.00, '2025-05-01', 'Cash')	Error Code: 3819. Check constraint 'chk_payments_amount' is violated.	0.000 sec
138	21:57:08	INSERT INTO payments VALUES (99, 1, 100000.00, '2025-05-01', 'Momo')	Error Code: 1265. Data truncated for column 'method' at row 1	0.000 sec
139	21:57:08	INSERT INTO repairers VALUES (99, 'Người mới', 'long.nh@suachua.vn', 'Test', 1)	Error Code: 1062. Duplicate entry 'long.nh@suachua.vn' for key 'repairers.uq_repairers_email'	0.000 sec
140	21:57:08	DELETE FROM customers WHERE customer_id = 1	Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails (`electronics_mini`.`s...	0.000 sec
141	21:57:08	SELECT COUNT(*) AS payments_rows FROM payments LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
142	21:57:08	ROLLBACK	0 row(s) affected	0.000 sec

check_name	violation_count
K5 thu tiền khi chưa có chi phí	1
K6 tổng thu vượt chi phí	1

3. Sample data

3.1. Record counts

Table 7. K1: record counts against the required minimum

Table	**Rows**	**Minimum**
specialties	6	5
repairers	8	6
customers	12	10
service_orders	22	12
service_reports	16	10

payments	18	10

3.2. Scenario matrix

Table 8. Required scenarios and the records that satisfy them

Required scenario	**Records**
Transactions in January 2025	Orders 1-5
Remaining months of Q1	Orders 6-8 (February), 9-11 (March)
Outside Q1 but within 2025	Orders 12-18
Outside 2025	Orders 19, 20 (2024); 21, 22 (2026)
All three statuses	Pending 10, 17, 22; Cancelled 4, 15, 18; the other 16 are Completed
Customer with more than 3 Completed orders in 2025	Customer 1 (orders 1, 3, 6, 13, 14)
Customer with orders but no payments	Customers 4, 5, 6, 11
Order paid in multiple instalments	Order 2 (3 payments), orders 13 and 16 (2 payments each)
Order with outstanding debt	Orders 6, 8, 21 (partially paid); order 7 (unpaid)
Specialty with no order in Q1	Specialty 5 (has a repairer), specialty 6 (has no repairer)
Repairer with no order in 2025	Repairer 8 (orders only in 2024 and 2026)

3.3. Additional edge cases

- **Time boundaries:** several orders are timestamped at or near period limits, namely order 5 (31/01/2025 23:30), order 11 (31/03/2025 23:59:59), order 12 (01/04/2025 00:00), order 18 (31/12/2025 23:59:59) and order 21 (01/01/2026 00:00).
- **Duplicate names:** customers 1 and 3 share the name Nguyễn Thị Hương, and repairers 2 and 5 share the name Trần Thị Mai. Consequently, all query outputs report the identifier alongside the name.
- **Ties:** repairers 1 and 4 are tied in Query 5, and customers 9 and 12 are tied in Query 8.
- **Near-threshold case:** customer 2 has exactly 3 Completed orders in 2025 and one further order in 2024, and therefore must not satisfy the condition of Query 10.
- **Payment year differing from order year:** payment 16 was made in 2025 for an order received in 2024, whereas payment 15 was made in 2026 for an order received in 2025.
- **Zero cost:** order 12 is a warranty repair with a recorded cost of 0.
- **Customer without orders:** customer 10 has placed no orders and has no contact details.

3.4. Validation results

Table 9. K2-K7: integrity checks (expected violation count: 0)

Check	Violations
K2 repairers → specialties	0
K2 service_orders → customers	0
K2 service_orders → repairers	0
K2 service_reports → service_orders	0
K2 payments → service_orders	0
K3 Non-Completed order with a report	0
K4 Completed order without exactly one report	0
K5 Payment without a confirmed cost	0
K6 Total paid exceeds cost	0
K7 Payment dated before the received date	0

Table 10. K8: scenario coverage

Scenario	Count
Orders in January 2025	5
Orders in February-March 2025	6
Orders in 2025 outside Q1	7
Orders outside 2025	4
Pending / Completed / Cancelled orders	3 / 16 / 3
Customers with more than 3 Completed orders in 2025	1
Customers with orders but no payment	4
Orders paid in several instalments	3
Orders with outstanding debt	4
Specialties with no order in Q1 2025	2
Repairers with no order in 2025	1

table_name	row_count	min_required
specialties	6	5
repairers	8	6
customers	12	10
service_orders	22	12
service_reports	16	10
payments	18	10

table_name	row_count	min_required
specialties	6	5
repairers	8	6
customers	12	10
service_orders	22	12
service_reports	16	10
payments	18	10

	check_name	violation_count
▶	K3 phiếu không Completed nhưng có biên bản	0
	K4 Completed không có đúng 1 biên bản	0
	K5 thu tiền khi chưa có chi phí	0
	K6 tổng thu vượt chi phí	0
	K7 thanh toán trước ngày nhận	0

	scenario	so_ban_ghi
▶	Q1/2025 có giao dịch tháng 01	5
	Q1/2025 có giao dịch tháng 02-03	6
	Năm 2025 ngoài quý I	7
	Ngoài năm 2025	4
	Trạng thái Pending	3
	Trạng thái Completed	16
	Trạng thái Cancelled	3
	Khách > 3 Completed năm 2025	1
	Khách có giao dịch nhưng chưa thanh toán	4
	Phiếu trả nhiều lần	3
	Phiếu còn nợ	4
	Nhóm chuyên môn 0 giao dịch Q1/2025	2
	Nhân viên 0 giao dịch năm 2025	1

3.5. Data modification test

To verify that the checks and queries respond to the underlying data rather than to hard-coded values, the timestamp of order 11 was changed from 31/03/2025 23:59:59 to 01/04/2025 00:00:00 within a transaction that was subsequently rolled back. As expected, the February-March count in K8 decreased from 6 to 5, the count of orders outside Q1 increased from 7 to 8, and the figure for specialty 2 in Query 2 decreased from 4 to 3.

	scenario	so_ban_ghi
▶	Q1/2025 có giao dịch tháng 01	5
	Q1/2025 có giao dịch tháng 02-03	5
	Năm 2025 ngoài quý I	8
	Ngoài năm 2025	4
	Trạng thái Pending	3
	Trạng thái Completed	16

specialty_id	specialty_name	total_orders_q1_2025
1	Máy tính & Laptop	4
2	Điện thoại di động	3
3	Máy tính bảng & Thiết bị đeo	2
4	Âm thanh & Điện tử gia dụng	1
5	Thiết bị mạng & Camera	0
6	Máy chơi game	0

4. Queries

Each query is documented according to a consistent structure: (i) the requirement and its potential pitfalls; (ii) a manual calculation derived from the raw data; (iii) the observed result; and (iv) a negative test, in which a deliberately incorrect variant demonstrates why the adopted logic is necessary.

4.1. Query 1: Completed orders in January 2025

Requirement and potential pitfalls. Only orders with status Completed are to be returned. The date range must cover the whole of 31 January, and since customers 1 and 3 share the same name, identifiers must be displayed to distinguish them.

```
SELECT
    so.order_id,
    so.received_at,
    so.device_name,
    c.customer_id,
    c.full_name  AS customer_name,
    r.repairer_id,
    r.full_name  AS repairer_name
FROM service_orders so
JOIN customers c ON c.customer_id = so.customer_id
JOIN repairers r ON r.repairer_id = so.repairer_id
WHERE so.status = 'Completed'
  AND so.received_at >= '2025-01-01'
  AND so.received_at <  '2025-02-01'
ORDER BY so.received_at, so.order_id;
```

Manual calculation. Orders 1-5 were received in January. Excluding order 4, which was Cancelled, leaves orders 1, 2, 3 and 5.

Observed result. The output is consistent with the manual calculation.

Table 11. Result of Query 1

order_id	received_at	Customer	Repairer
1	2025-01-03 08:30:00	1 - Nguyễn Thị Hương	1 - Nguyễn Hoàng Long
2	2025-01-08 10:15:00	7 - Vũ Thị Lan	3 - Lê Văn Đức
3	2025-01-15 14:00:00	1 - Nguyễn Thị Hương	2 - Trần Thị Mai
5	2025-01-31 23:30:00	2 - Lê Minh Tuấn	3 - Lê Văn Đức

order_id	received_at	device_name	customer_id	customer_name	repairer_id	repairer_name
1	2025-01-03 08:30:00	Laptop Dell Inspiron 15	1	Nguyễn Thị Hương	1	Nguyễn Hoàng Long
2	2025-01-08 10:15:00	iPhone 13 Pro	7	Vũ Thị Lan	3	Lê Văn Đức
3	2025-01-15 14:00:00	PC Asus ROG Strix	1	Nguyễn Thị Hương	2	Trần Thị Mai
5	2025-01-31 23:30:00	Samsung Galaxy A54	2	Lê Minh Tuấn	3	Lê Văn Đức

Negative test. Using `BETWEEN '2025-01-01' AND '2025-01-31'` returns only 3 rows, because the upper bound is interpreted as 31/01 00:00:00 and order 5 is consequently excluded. Removing the status filter returns 5 rows, including the Cancelled order 4.

4.2. Query 2: Orders per specialty in Q1 2025

Requirement and potential pitfalls. Orders of all statuses are counted, and specialties with no orders must still be displayed. This is relevant because specialty 6 has no repairer and specialty 5 received no orders in Q1.

```
SELECT
    s.specialty_id,
    s.name               AS specialty_name,
    COUNT(so.order_id)   AS total_orders_q1_2025
FROM specialties s
LEFT JOIN repairers r
       ON r.specialty_id = s.specialty_id
LEFT JOIN service_orders so
       ON so.repairer_id = r.repairer_id
      AND so.received_at >= '2025-01-01'
      AND so.received_at <  '2025-04-01'
GROUP BY s.specialty_id, s.name
ORDER BY s.specialty_id;
```

Manual calculation. Repairers 1 and 2 belong to specialty 1; repairers 3 and 4 to specialty 2; repairer 5 to specialty 3; repairers 6 and 8 to specialty 4; and repairer 7 to specialty 5. Mapping orders 1-11 accordingly yields specialty 1 = {1, 3, 4, 8}, specialty 2 = {2, 5, 7, 11}, specialty 3 = {6, 10} and specialty 4 = {9}.

Observed result. The output is consistent with the manual calculation.

Table 12. Result of Query 2

specialty_id	Name	Orders in Q1 2025
1	Máy tính & Laptop	4
2	Điện thoại di động	4
3	Máy tính bảng & Thiết bị đeo	2
4	Âm thanh & Điện tử gia dụng	1
5	Thiết bị mạng & Camera	0
6	Máy chơi game	0

specialty_id	specialty_name	total_orders_q1_2025
1	Máy tính & Laptop	4
2	Điện thoại di động	4
3	Máy tính bảng & Thiết bị đeo	2
4	Âm thanh & Điện tử gia dụng	1
5	Thiết bị mạng & Camera	0
6	Máy chơi game	0

Negative test.

- Moving the date filter from the `ON` clause to the `WHERE` clause eliminates specialties 5 and 6, since their NULL `received_at` values fail the comparison.
- Replacing `COUNT(so.order_id)` with `COUNT(*)` yields 1 for specialties 5 and 6, and 2 instead of 1 for specialty 4, because repairer 8, who has no Q1 orders, still generates a NULL row.
- Setting the upper bound to `'2025-03-31'` excludes order 11, reducing the count for specialty 2 to 3.

4.3. Query 3: Customers with an order lacking a report

Requirement and potential pitfalls. Cancelled orders are excluded, and each customer must appear only once. As no time restriction applies, orders from 2026 are also taken into account.

```
SELECT
    c.customer_id,
    c.full_name AS customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM service_orders so
    LEFT JOIN service_reports sr ON sr.order_id = so.order_id
    WHERE so.customer_id = c.customer_id
      AND so.status <> 'Cancelled'
```

```
      AND sr.report_id IS NULL
)
ORDER BY c.customer_id;
```

Manual calculation. The orders that are neither Cancelled nor associated with a report are the Pending orders 10 (customer 5), 17 (customer 4) and 22 (customer 9).

Observed result. The query returns customers 4 Hoàng Văn Nam, 5 Đỗ Thu Trang and 9 Mai Thanh Hà, which is consistent with the manual calculation.

	customer_id	customer_name
▶	4	Hoàng Văn Nam
	5	Đỗ Thu Trang
	9	Mai Thanh Hà
*	NULL	NULL

Negative test. Removing the condition `status <> 'Cancelled'` adds customers 3, 6 and 7, yielding 6 rows. Since the sample contains no customer with two open orders, a temporary Pending order was inserted for customer 4 within a rolled-back transaction. Under this condition, a plain `JOIN` listed customer 4 twice, whereas `EXISTS` listed the customer only once.

4.4. Query 4: Average repair cost per specialty

Requirement and potential pitfalls. The average is computed per Completed order across the entire period, and only specialties with recorded costs are displayed. Despite its zero cost, order 12 remains a Completed transaction and is therefore retained in the denominator.

```
SELECT
    s.specialty_id,
    s.name                      AS specialty_name,
    COUNT(sr.report_id)         AS completed_orders,
    ROUND(AVG(sr.cost), 2)      AS avg_cost_per_completed
FROM specialties s
JOIN repairers r        ON r.specialty_id = s.specialty_id
JOIN service_orders so  ON so.repairer_id = r.repairer_id
                       AND so.status = 'Completed'
JOIN service_reports sr ON sr.order_id = so.order_id
GROUP BY s.specialty_id, s.name
HAVING SUM(sr.cost) > 0
ORDER BY s.specialty_id;
```

Manual calculation. Specialty 1: orders 1, 3, 8, 12, 16 and 20 total 13,300,000 across 6 orders. Specialty 2: orders 2, 5, 7, 11 and 13 total 9,250,000 across 5 orders. Specialty 3: order 6, at

1,400,000. Specialty 4: orders 9, 19 and 21 total 4,600,000 across 3 orders. Specialty 5: order 14, at 550,000. Specialty 6 has no recorded cost.

Observed result. The output is consistent with the manual calculation.

Table 13. Result of Query 4

specialty_id	Name	Completed	Average cost
1	Máy tính & Laptop	6	2,216,666.67
2	Điện thoại di động	5	1,850,000.00
3	Máy tính bảng & Thiết bị đeo	1	1,400,000.00
4	Âm thanh & Điện tử gia dụng	3	1,533,333.33
5	Thiết bị mạng & Camera	1	550,000.00

	specialty_id	specialty_name	completed_orders	avg_cost_per_completed
▶	1	Máy tính & Laptop	6	2216666.67
	2	Điện thoại di động	5	1850000.00
	3	Máy tính bảng & Thiết bị đeo	1	1400000.00
	4	Âm thanh & Điện tử gia dụng	3	1533333.33
	5	Thiết bị mạng & Camera	1	550000.00

Negative test. Adding the filter `WHERE sr.cost > 0` excludes order 12 and inflates the average for specialty 1 to 2,660,000.00. Joining `payments` duplicates the cost of orders paid in instalments and thereby distorts the averages: specialty 1 becomes 2,200,000.00 and specialty 2 becomes 2,281,250.00.

4.5. Query 5: Top 3 repairers in 2025

Requirement and potential pitfalls. Completed orders received in 2025 are counted; ties are resolved in favour of the lower `repairer_id`; and only repairers with at least one qualifying order are eligible.

```
SELECT
    r.repairer_id,
    r.full_name          AS repairer_name,
    COUNT(so.order_id)   AS completed_orders_2025
FROM repairers r
JOIN service_orders so ON so.repairer_id = r.repairer_id
```

```
WHERE so.status = 'Completed'
  AND so.received_at >= '2025-01-01'
  AND so.received_at <  '2026-01-01'
GROUP BY r.repairer_id, r.full_name
ORDER BY completed_orders_2025 DESC, r.repairer_id ASC
LIMIT 3;
```

Manual calculation. Repairer 2: orders 3, 8 and 16 (3). Repairer 3: orders 2, 5 and 13 (3). Repairer 1: orders 1 and 12 (2). Repairer 4: orders 7 and 11 (2). Repairers 5, 6 and 7: 1 order each. As repairers 1 and 4 are tied, repairer 1 takes third place.

Observed result. Repairer 2 Trần Thị Mai (3), repairer 3 Lê Văn Đức (3) and repairer 1 Nguyễn Hoàng Long (2), which is consistent with the manual calculation.

	repairer_id	repairer_name	completed_orders_2025
▶	2	Trần Thị Mai	3
	3	Lê Văn Đức	3
	1	Nguyễn Hoàng Long	2

Negative test. Applying the tie-break in descending order returns repairers 3, 2 and 4, altering both the ranking order and the repairer in third place.

4.6. Query 6: Total paid per customer in 2025

Requirement and potential pitfalls. Payments must be filtered by `payment_date` rather than `received_at`, and customers who made no payment must be displayed with a total of 0.

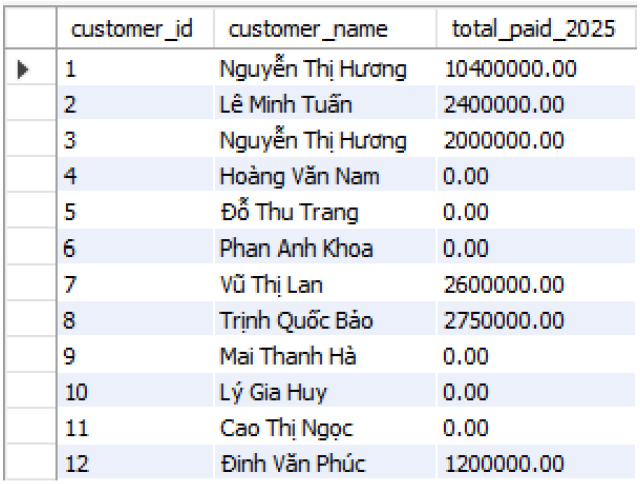
```
SELECT
    c.customer_id,
    c.full_name                          AS customer_name,
    COALESCE(pay.total_paid_2025, 0.00)  AS total_paid_2025
FROM customers c
LEFT JOIN (
    SELECT so.customer_id, SUM(p.amount) AS total_paid_2025
    FROM payments p
    JOIN service_orders so ON so.order_id = p.order_id
    WHERE p.payment_date >= '2025-01-01'
      AND p.payment_date <  '2026-01-01'
    GROUP BY so.customer_id
) pay ON pay.customer_id = c.customer_id
ORDER BY c.customer_id;
```

Manual calculation. Customer 2 constitutes the critical case: the payments of 750,000, 650,000 and 1,000,000 fall within 2025, whereas those of 1,100,000 (2026) and 1,650,000 (2024) do not, yielding a total of 2,400,000. Customer 8 paid 2,750,000 in 2025 for an order received in 2024. Customer 9 made payments only in 2026, and the total is therefore 0.

Observed result. The output is consistent with the manual calculation.

Table 14. Result of Query 6

customer_id	Name	Paid in 2025
1	Nguyễn Thị Hương	10,400,000.00
2	Lê Minh Tuấn	2,400,000.00
3	Nguyễn Thị Hương	2,000,000.00
4	Hoàng Văn Nam	0.00
5	Đỗ Thu Trang	0.00
6	Phan Anh Khoa	0.00
7	Vũ Thị Lan	2,600,000.00
8	Trịnh Quốc Bảo	2,750,000.00
9	Mai Thanh Hà	0.00
10	Lý Gia Huy	0.00
11	Cao Thị Ngọc	0.00
12	Đinh Văn Phúc	1,200,000.00



	customer_id	customer_name	total_paid_2025
▶	1	Nguyễn Thị Hương	10400000.00
	2	Lê Minh Tuấn	2400000.00
	3	Nguyễn Thị Hương	2000000.00
	4	Hoàng Văn Nam	0.00
	5	Đỗ Thu Trang	0.00
	6	Phan Anh Khoa	0.00
	7	Vũ Thị Lan	2600000.00
	8	Trịnh Quốc Bảo	2750000.00
	9	Mai Thanh Hà	0.00
	10	Lý Gia Huy	0.00
	11	Cao Thị Ngọc	0.00
	12	Đinh Văn Phúc	1200000.00

Negative test. Filtering by `received_at` changes the total for customer 2 to 3,500,000 and for customer 8 to 0. Placing the date filter in the `WHERE` clause of a `LEFT JOIN` retains only the 6 customers who made payments.

4.7. Query 7: Customers with no payment record

Requirement and potential pitfalls. The condition is evaluated at the customer level rather than the order level: a customer with one unpaid order but another paid order does not qualify. Customers who have placed no orders must also be included.

```
SELECT
    c.customer_id,
    c.full_name AS customer_name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM service_orders so
    JOIN payments p ON p.order_id = so.order_id
    WHERE so.customer_id = c.customer_id
)
ORDER BY c.customer_id;
```

Manual calculation. Customers with at least one payment are 1, 2, 3, 7, 8, 9 and 12, leaving customers 4, 5, 6, 10 and 11.

Observed result. The query returns customers 4 Hoàng Văn Nam, 5 Đỗ Thu Trang, 6 Phan Anh Khoa, 10 Lý Gia Huy and 11 Cao Thị Ngọc, which is consistent with the manual calculation.

customer_id	customer_name
4	Hoàng Văn Nam
5	Đỗ Thu Trang
6	Phan Anh Khoa
10	Lý Gia Huy
11	Cao Thị Ngọc
NULL	NULL

Negative test. A row-level `LEFT JOIN ... WHERE p.payment_id IS NULL` incorrectly includes customers 3, 7 and 9, each of whom has paid for another order. In addition, an inner join to the orders table omits customer 10.

4.8. Query 8: Customers by total confirmed cost

Requirement and potential pitfalls. All customers must appear, the total is independent of payments, and ties are ordered by customer identifier. The `payments` table must not be joined in this query, in order to avoid double counting.

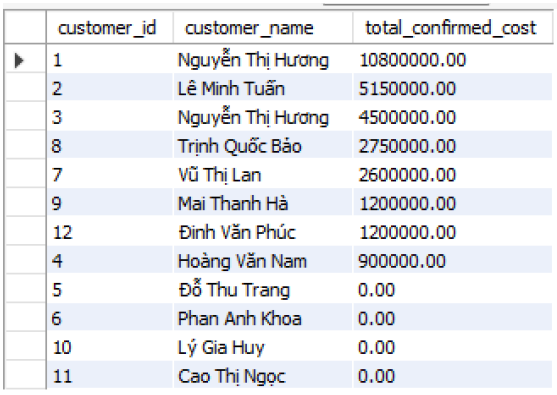
```
SELECT
    c.customer_id,
    c.full_name                     AS customer_name,
    COALESCE(SUM(sr.cost), 0.00)    AS total_confirmed_cost
FROM customers c
LEFT JOIN service_orders so  ON so.customer_id = c.customer_id
LEFT JOIN service_reports sr ON sr.order_id = so.order_id
GROUP BY c.customer_id, c.full_name
ORDER BY total_confirmed_cost DESC, c.customer_id ASC;
```

Manual calculation. Customer 1: 1,850,000 + 3,200,000 + 1,400,000 + 3,800,000 + 550,000 = 10,800,000. Customer 2: 750,000 + 650,000 + 2,100,000 + 1,650,000 = 5,150,000. Customers 9 and 12 are tied at 1,200,000, so customer 9 is listed first.

Observed result. The output is consistent with the manual calculation.

Table 15. Result of Query 8

customer_id	Name	Total confirmed cost
1	Nguyễn Thị Hương	10,800,000.00
2	Lê Minh Tuấn	5,150,000.00
3	Nguyễn Thị Hương	4,500,000.00
8	Trịnh Quốc Bảo	2,750,000.00
7	Vũ Thị Lan	2,600,000.00
9	Mai Thanh Hà	1,200,000.00
12	Đinh Văn Phúc	1,200,000.00
4	Hoàng Văn Nam	900,000.00
5	Đỗ Thu Trang	0.00
6	Phan Anh Khoa	0.00
10	Lý Gia Huy	0.00
11	Cao Thị Ngọc	0.00



customer_id	customer_name	total_confirmed_cost
1	Nguyễn Thị Hương	10800000.00
2	Lê Minh Tuấn	5150000.00
3	Nguyễn Thị Hương	4500000.00
8	Trịnh Quốc Bảo	2750000.00
7	Vũ Thị Lan	2600000.00
9	Mai Thanh Hà	1200000.00
12	Đinh Văn Phúc	1200000.00
4	Hoàng Văn Nam	900000.00
5	Đỗ Thu Trang	0.00
6	Phan Anh Khoa	0.00
10	Lý Gia Huy	0.00
11	Cao Thị Ngọc	0.00

Negative test. Introducing a join to `payments` inflates the totals for customer 1 to 14,600,000, customer 7 to 7,800,000 and customer 2 to 7,250,000. As a result, customer 7 is incorrectly ranked above customer 2.

4.9. Query 9: Cancellation rate per repairer in 2025

Requirement and potential pitfalls. The cancellation rate is defined as the number of Cancelled orders divided by the total number of orders in 2025, multiplied by 100 and rounded to 2 decimal places; it is NULL when the total is 0. Repairer 8 must be retained in the result.

```
SELECT
    r.repairer_id,
      r.full_name                                                    AS
repairer_name,
      COUNT(so.order_id)                                             AS
total_orders_2025,
        SUM(CASE  WHEN  so.status  =  'Cancelled'  THEN  1  ELSE  0  END)      AS
cancelled_orders_2025,
    ROUND(100.0 * SUM(CASE WHEN so.status = 'Cancelled' THEN 1 ELSE 0 END)
                        /  NULLIF(COUNT(so.order_id),  0),  2)               AS
cancellation_rate_pct
FROM repairers r
LEFT JOIN service_orders so
       ON so.repairer_id = r.repairer_id
      AND so.received_at >= '2025-01-01'
      AND so.received_at <  '2026-01-01'
GROUP BY r.repairer_id, r.full_name
ORDER BY r.repairer_id;
```

Manual calculation. Repairer 1: one cancellation (order 4) among orders 1, 4 and 12, i.e. 1/3. Repairer 4: one cancellation (order 18) among orders 7, 11 and 18, i.e. 1/3. Repairer 6: one cancellation (order 15) among orders 9 and 15, i.e. 1/2. Repairer 8 has no orders in 2025.

Observed result. The output is consistent with the manual calculation.

Table 16. Result of Query 9

repairer_id	Name	Total	Cancelled	Rate (%)
1	Nguyễn Hoàng Long	3	1	33.33
2	Trần Thị Mai	3	0	0.00
3	Lê Văn Đức	3	0	0.00
4	Phạm Quốc Tuấn	3	1	33.33

5	Trần Thị Mai	2	0	0.00
6	Vũ Đình Khánh	2	1	50.00
7	Đặng Minh Quân	2	0	0.00
8	Bùi Thanh Sơn	0	0	NULL

	repairer_id	repairer_name	total_orders_2025	cancelled_orders_2025	cancellation_rate_pct
▶	1	Nguyễn Hoàng Long	3	1	33.33
	2	Trần Thị Mai	3	0	0.00
	3	Lê Văn Đức	3	0	0.00
	4	Phạm Quốc Tuấn	3	1	33.33
	5	Trần Thị Mai	2	0	0.00
	6	Vũ Đình Khánh	2	1	50.00
	7	Đặng Minh Quân	2	0	0.00
	8	Bùi Thanh Sơn	0	0	NULL

Negative test.

- Without `NULLIF`, repairer 8 still displays NULL, because MySQL returns NULL for division by zero within a `SELECT` statement; however, warning 1365 is raised. `NULLIF` makes the intended behaviour explicit and suppresses the warning.
- `COUNT(*)` counts the NULL row as one order, so repairer 8 displays 0.00 instead of NULL.
- Moving the date filter to the `WHERE` clause eliminates repairer 8 from the result.

4.10. Query 10: Customers with more than 3 Completed orders in 2025

Requirement and potential pitfalls. Because the condition applies to aggregated groups, a `HAVING` clause is required. The threshold is strictly greater than 3.

```
SELECT
    c.customer_id,
    c.full_name          AS customer_name,
    COUNT(so.order_id)   AS completed_orders_2025
FROM customers c
JOIN service_orders so ON so.customer_id = c.customer_id
WHERE so.status = 'Completed'
  AND so.received_at >= '2025-01-01'
  AND so.received_at <  '2026-01-01'
GROUP BY c.customer_id, c.full_name
HAVING COUNT(so.order_id) > 3
ORDER BY completed_orders_2025 DESC, c.customer_id ASC;
```

Manual calculation. Customer 1 has 5 Completed orders in 2025 (orders 1, 3, 6, 13 and 14). Customer 2 has 3 in 2025 (orders 5, 9 and 16), since order 20 dates from 2024.

Observed result. Only customer 1 Nguyễn Thị Hương is returned, with 5 orders, which is consistent with the manual calculation.

	customer_id	customer_name	completed_orders_2025
▶	1	Nguyễn Thị Hương	5

Negative test. Replacing `> 3` with `>= 3` adds customer 2 with 3 orders. Removing the year filter likewise adds customer 2, in this case with 4 orders.

5. Views

All views span the entire time range of the dataset and are not restricted to 2025.

5.1. v_detail

Each row of the view represents a single service order. A `LEFT JOIN` to `service_reports` preserves orders without a report, for which the cost is NULL. Since each order has at most one report, the join cannot produce duplicate rows.

```
CREATE OR REPLACE VIEW v_detail AS
SELECT
    so.order_id,
    so.received_at,
    so.status,
    so.device_name,
    so.serial_number,
    c.customer_id,
    c.full_name    AS customer_name,
    r.repairer_id,
    r.full_name    AS repairer_name,
    s.specialty_id,
    s.name         AS specialty_name,
    sr.cost        AS repair_cost
FROM service_orders so
JOIN customers   c ON c.customer_id  = so.customer_id
JOIN repairers   r ON r.repairer_id  = so.repairer_id
JOIN specialties s ON s.specialty_id = r.specialty_id
LEFT JOIN service_reports sr ON sr.order_id = so.order_id;
```

Table 17. Result of `v_detail` (device and serial number columns omitted)

Order	Received	Status	Customer	Repairer	Spec.	Cost
1	2025-01-03	Completed	1 Nguyễn Thị Hương	1 Nguyễn Hoàng Long	1	1,850,000
2	2025-01-08	Completed	7 Vũ Thị Lan	3 Lê Văn Đức	2	2,600,000
3	2025-01-15	Completed	1 Nguyễn Thị Hương	2 Trần Thị Mai	1	3,200,000
4	2025-01-22	Cancelled	6 Phan Anh Khoa	1 Nguyễn Hoàng Long	1	NULL
5	2025-01-31	Completed	2 Lê Minh Tuấn	3 Lê Văn Đức	2	750,000
6	2025-02-05	Completed	1 Nguyễn Thị Hương	5 Trần Thị Mai	3	1,400,000
7	2025-02-14	Completed	4 Hoàng Văn Nam	4 Phạm Quốc Tuấn	2	900,000
8	2025-02-27	Completed	3 Nguyễn Thị Hương	2 Trần Thị Mai	1	4,500,000
9	2025-03-10	Completed	2 Lê Minh Tuấn	6 Vũ Đình Khánh	4	650,000
10	2025-03-18	Pending	5 Đỗ Thu Trang	5 Trần Thị Mai	3	NULL
11	2025-03-31	Completed	12 Đinh Văn Phúc	4 Phạm Quốc Tuấn	2	1,200,000
12	2025-04-01	Completed	11 Cao Thị Ngọc	1 Nguyễn Hoàng Long	1	0
13	2025-05-12	Completed	1 Nguyễn Thị Hương	3 Lê Văn Đức	2	3,800,000
14	2025-06-20	Completed	1 Nguyễn Thị Hương	7 Đặng Minh Quân	5	550,000
15	2025-08-08	Cancelled	7 Vũ Thị Lan	6 Vũ Đình Khánh	4	NULL
16	2025-09-25	Completed	2 Lê Minh Tuấn	2 Trần Thị Mai	1	2,100,000
17	2025-11-11	Pending	4 Hoàng Văn Nam	7 Đặng Minh Quân	5	NULL
18	2025-12-31	Cancelled	3 Nguyễn Thị Hương	4 Phạm Quốc Tuấn	2	NULL
19	2024-11-20	Completed	8 Trịnh Quốc Bảo	8 Bùi Thanh Sơn	4	2,750,000
20	2024-12-28	Completed	2 Lê Minh Tuấn	1 Nguyễn Hoàng Long	1	1,650,000
21	2026-01-01	Completed	9 Mai Thanh Hà	8 Bùi Thanh Sơn	4	1,200,000
22	2026-02-10	Pending	9 Mai Thanh Hà	3 Lê Văn Đức	2	NULL



order_id	received_at	status	device_name	serial_number	customer_id	customer_name	repairer_id	repairer_name	specialty_id	specialty_name	repair_cost
1	2025-01-03 08:30:00	Completed	Laptop Dell Inspiron 15	DL-INS15-7781	1	Nguyễn Thị Hương	1	Nguyễn Hoàng Long	1	Máy tính & Laptop	1850000.00
2	2025-01-08 10:15:00	Completed	iPhone 13 Pro	AP-IP13P-2210	7	Vũ Thị Lan	3	Lê Văn Đức	2	Điện thoại di động	2600000.00
3	2025-01-15 14:00:00	Completed	PC Asus ROG Strix	AS-ROG-5531	1	Nguyễn Thị Hương	2	Trần Thị Mai	1	Máy tính & Laptop	3200000.00
4	2025-01-22 16:45:00	Cancelled	Laptop HP Envy x360	HP-ENV-0912	6	Phan Anh Khoa	1	Nguyễn Hoàng Long	1	Máy tính & Laptop	NULL
5	2025-01-31 23:30:00	Completed	Samsung Galaxy A54	SS-A54-6618	2	Lê Minh Tuấn	3	Lê Văn Đức	2	Điện thoại di động	750000.00
6	2025-02-05 09:00:00	Completed	iPad Gen 9	AP-IPD9-4402	1	Nguyễn Thị Hương	5	Trần Thị Mai	3	Máy tính bảng & Thiết bị đeo	1400000.00
7	2025-02-14 11:20:00	Completed	Xiaomi Redmi Note 12	XM-RN12-3307	4	Hoàng Văn Nam	4	Phạm Quốc Tuấn	2	Điện thoại di động	900000.00
8	2025-02-27 15:10:00	Completed	MacBook Pro 14 M3	AP-MBP14-8820	3	Nguyễn Thị Hương	2	Trần Thị Mai	1	Máy tính & Laptop	4500000.00
9	2025-03-10 13:00:00	Completed	Loa JBL Charge 5	JBL-CH5-1145	2	Lê Minh Tuấn	6	Vũ Đình Khánh	4	Âm thanh & Điện tử gia dụng	650000.00
10	2025-03-18 10:30:00	Pending	Apple Watch Series 8	AP-AW8-7719	5	Đỗ Thu Trang	5	Trần Thị Mai	3	Máy tính bảng & Thiết bị đeo	NULL
11	2025-03-31 23:59:59	Completed	OPPO Reno 10	OP-RN10-5520	12	Đinh Văn Phúc	4	Phạm Quốc Tuấn	2	Điện thoại di động	1200000.00
12	2025-04-01 00:00:00	Completed	Laptop Lenovo IdeaP...	LN-IDP5-2290	11	Cao Thị Ngọc	1	Nguyễn Hoàng Long	1	Máy tính & Laptop	0.00
13	2025-05-12 09:45:00	Completed	iPhone 15	AP-IP15-9031	1	Nguyễn Thị Hương	3	Lê Văn Đức	2	Điện thoại di động	3800000.00
14	2025-06-20 14:20:00	Completed	Camera Imou Ranger 2	IM-RG2-6604	1	Nguyễn Thị Hương	7	Đặng Minh Quân	5	Thiết bị mạng & Camera	550000.00
15	2025-08-08 10:00:00	Cancelled	Tivi Sony Bravia 43	SN-BRV43-3381	7	Vũ Thị Lan	6	Vũ Đình Khánh	4	Âm thanh & Điện tử gia dụng	NULL
16	2025-09-25 08:15:00	Completed	Laptop Acer Aspire 7	AC-ASP7-4417	2	Lê Minh Tuấn	2	Trần Thị Mai	1	Máy tính & Laptop	2100000.00
17	2025-11-11 16:00:00	Pending	Router TP-Link Archer...	TP-AC6-8852	4	Hoàng Văn Nam	7	Đặng Minh Quân	5	Thiết bị mạng & Camera	NULL
18	2025-12-31 23:59:59	Cancelled	Vivo Y36	VV-Y36-1203	3	Nguyễn Thị Hương	4	Phạm Quốc Tuấn	2	Điện thoại di động	NULL
19	2024-11-20 10:30:00	Completed	Ampli Denon PMA-600NE	DN-PMA6-0071	8	Trịnh Quốc Bảo	8	Bùi Thanh Sơn	4	Âm thanh & Điện tử gia dụng	2750000.00
20	2024-12-28 15:00:00	Completed	Laptop MSI Modern 14	MS-MD14-3326	2	Lê Minh Tuấn	1	Nguyễn Hoàng Long	1	Máy tính & Laptop	1650000.00
21	2026-01-01 00:00:00	Completed	Loa Marshall Emberton II	MR-EMB2-9914	9	Mai Thanh Hà	8	Bùi Thanh Sơn	4	Âm thanh & Điện tử gia dụng	1200000.00
22	2026-02-10 11:00:00	Pending	Samsung Galaxy Z Flip5	SS-ZF5-2268	9	Mai Thanh Hà	3	Lê Văn Đức	2	Điện thoại di động	NULL

Negative test. Replacing the `LEFT JOIN` with an inner `JOIN` returns 16 rows instead of 22, as the 3 Pending and 3 Cancelled orders are omitted.

5.2. v_group_summary

Each row represents a single specialty. The query takes `specialties` as its base table and applies `LEFT JOIN` operations, so that specialties without orders or repairers are retained with totals of 0.

```
CREATE OR REPLACE VIEW v_group_summary AS
SELECT
    s.specialty_id,
     s.name                                                                AS
specialty_name,
     COUNT(so.order_id)                                                    AS
total_orders,
       SUM(CASE  WHEN  so.status  =  'Completed'  THEN  1  ELSE  0  END)        AS
completed_orders,
       SUM(CASE  WHEN  so.status  =  'Cancelled'  THEN  1  ELSE  0  END)        AS
cancelled_orders,
     COALESCE(SUM(sr.cost), 0.00)                                          AS
total_repair_cost
FROM specialties s
LEFT JOIN repairers r        ON r.specialty_id = s.specialty_id
LEFT JOIN service_orders so  ON so.repairer_id = r.repairer_id
LEFT JOIN service_reports sr ON sr.order_id    = so.order_id
GROUP BY s.specialty_id, s.name;
```

Manual calculation. Specialty 1: orders 1, 3, 4, 8, 12, 16 and 20. Specialty 2: orders 2, 5, 7, 11, 13, 18 and 22. Specialty 3: orders 6 and 10. Specialty 4: orders 9, 15, 19 and 21. Specialty 5: orders 14 and 17. Specialty 6: none. The output of the view is consistent with these figures.

Table 18. Result of `v_group_summary`

ID	Name	Total	Completed	Cancelled	Total cost
1	Máy tính & Laptop	7	6	1	13,300,000.00
2	Điện thoại di động	7	5	1	9,250,000.00
3	Máy tính bảng & Thiết bị đeo	2	1	0	1,400,000.00
4	Âm thanh & Điện tử gia dụng	4	3	1	4,600,000.00
5	Thiết bị mạng & Camera	2	1	0	550,000.00
6	Máy chơi game	0	0	0	0.00

specialty_id	specialty_name	total_orders	completed_orders	cancelled_orders	total_repair_cost
1	Máy tính & Laptop	7	6	1	13300000.00
2	Điện thoại di động	7	5	1	9250000.00
3	Máy tính bảng & Thiết bị đeo	2	1	0	1400000.00
4	Âm thanh & Điện tử gia dụng	4	3	1	4600000.00
5	Thiết bị mạng & Camera	2	1	0	550000.00
6	Máy chơi game	0	0	0	0.00

Negative test. Using `COUNT(*)` assigns specialty 6 a total of 1 instead of 0.

5.3. v_debt

Each row represents a single customer. Outstanding debt is defined as the total confirmed cost minus the total amount actually paid. Costs and payments are first aggregated per customer in two separate subqueries and only then joined, so that each subquery contributes at most one row per customer.

```
CREATE OR REPLACE VIEW v_debt AS
SELECT
    c.customer_id,
    c.full_name                                 AS customer_name,
    COALESCE(cost_sum.total_cost, 0.00)         AS total_confirmed_cost,
    COALESCE(paid_sum.total_paid, 0.00)         AS total_paid,
    COALESCE(cost_sum.total_cost, 0.00)
      - COALESCE(paid_sum.total_paid, 0.00)     AS outstanding_debt
FROM customers c
LEFT JOIN (
    SELECT so.customer_id, SUM(sr.cost) AS total_cost
    FROM service_orders so
    JOIN service_reports sr ON sr.order_id = so.order_id
    GROUP BY so.customer_id
) cost_sum ON cost_sum.customer_id = c.customer_id
LEFT JOIN (
    SELECT so.customer_id, SUM(p.amount) AS total_paid
    FROM service_orders so
    JOIN payments p ON p.order_id = so.order_id
    GROUP BY so.customer_id
) paid_sum ON paid_sum.customer_id = c.customer_id;
```

Table 19. Result of `v_debt`

ID	Name	Confirmed cost	Paid	Debt
1	Nguyễn Thị Hương	10,800,000.00	10,400,000.00	400,000.00

2	Lê Minh Tuấn	5,150,000.00	5,150,000.00	0.00
3	Nguyễn Thị Hương	4,500,000.00	2,000,000.00	2,500,000.00
4	Hoàng Văn Nam	900,000.00	0.00	900,000.00
5	Đỗ Thu Trang	0.00	0.00	0.00
6	Phan Anh Khoa	0.00	0.00	0.00
7	Vũ Thị Lan	2,600,000.00	2,600,000.00	0.00
8	Trịnh Quốc Bảo	2,750,000.00	2,750,000.00	0.00
9	Mai Thanh Hà	1,200,000.00	500,000.00	700,000.00
10	Lý Gia Huy	0.00	0.00	0.00
11	Cao Thị Ngọc	0.00	0.00	0.00
12	Đinh Văn Phúc	1,200,000.00	1,200,000.00	0.00

customer_id	customer_name	total_confirmed_cost	total_paid	outstanding_debt
1	Nguyễn Thị Hương	10800000.00	10400000.00	400000.00
2	Lê Minh Tuấn	5150000.00	5150000.00	0.00
3	Nguyễn Thị Hương	4500000.00	2000000.00	2500000.00
4	Hoàng Văn Nam	900000.00	0.00	900000.00
5	Đỗ Thu Trang	0.00	0.00	0.00
6	Phan Anh Khoa	0.00	0.00	0.00
7	Vũ Thị Lan	2600000.00	2600000.00	0.00
8	Trịnh Quốc Bảo	2750000.00	2750000.00	0.00
9	Mai Thanh Hà	1200000.00	500000.00	700000.00
10	Lý Gia Huy	0.00	0.00	0.00
11	Cao Thị Ngọc	0.00	0.00	0.00
12	Đinh Văn Phúc	1200000.00	1200000.00	0.00

5.4. The double-counting pitfall

To demonstrate the necessity of these subqueries, an incorrect variant that joins `service_reports` and `payments` directly was tested. In this variant, an order with n payments appears in n rows, each carrying the full cost of the order, which inflates the aggregated cost.

```
SELECT c.customer_id,
       COALESCE(SUM(sr.cost), 0) AS cost,
       COALESCE(SUM(p.amount), 0) AS paid,
       COALESCE(SUM(sr.cost), 0) - COALESCE(SUM(p.amount), 0) AS debt
FROM customers c
LEFT JOIN service_orders so  ON so.customer_id = c.customer_id
LEFT JOIN service_reports sr ON sr.order_id = so.order_id
LEFT JOIN payments p         ON p.order_id = so.order_id
GROUP BY c.customer_id;
```

Table 20. Correct and wrong `v_debt` for the affected customers

Customer	Correct cost	Wrong cost	Correct debt	Wrong debt	Cause
1	10,800,000	14,600,000	400,000	4,200,000	Order 13 has 2 payments
2	5,150,000	7,250,000	0	2,100,000	Order 16 has 2 payments
7	2,600,000	7,800,000	0	5,200,000	Order 2 has 3 payments

customer_id	correct_cost	wrong_cost	correct_debt	wrong_debt
1	10800000.00	14600000.00	400000.00	4200000.00
2	5150000.00	7250000.00	0.00	2100000.00
7	2600000.00	7800000.00	0.00	5200000.00

6. Conclusion

The four SQL scripts execute in the prescribed order on a newly created database and address Parts A to D of the assignment. The schema enforces all single-table and referential constraints, as confirmed by tests V1-V10. The sample dataset covers every required scenario, and the outputs of all ten queries and three views are consistent with manual calculations derived from the raw data.

Nevertheless, the work is subject to the following limitations:

- Cross-table business rules are detected by queries rather than prevented at the time of insertion, as demonstrated by V11 and V12. Although triggers could enforce these rules, they were not implemented because the assignment does not require them.
- The sample dataset contains no customer with two open orders; consequently, the once-per-customer behaviour of Query 3 was demonstrated using a temporary row rather than the permanent data.